

CUSTOMER SERVICE REQUEST ANALYSIS

NYC 311 Dataset Project

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats
```

```
In [2]: # The 'r' before the quotes is critical for Windows paths
df = pd.read_csv("311_Service_Requests_from_2010_to_Present.csv", low_memory=False)
```

```
In [3]: # 1.2 & 1.3 Visualize dataset and print columns
print("--- First 5 Rows ---")
print(df.head())
print("\n--- DataFrame Columns ---")
print(df.columns)
```

--- First 5 Rows ---

	Unique Key	Created Date	Closed Date	Agency \
0	32310363	12/31/2015 11:59:45 PM	01/01/2016 12:55:15 AM	NYPD
1	32309934	12/31/2015 11:59:44 PM	01/01/2016 01:26:57 AM	NYPD
2	32309159	12/31/2015 11:59:29 PM	01/01/2016 04:51:03 AM	NYPD
3	32305098	12/31/2015 11:57:46 PM	01/01/2016 07:43:13 AM	NYPD
4	32306529	12/31/2015 11:56:58 PM	01/01/2016 03:24:42 AM	NYPD

	Agency Name	Complaint Type \
0	New York City Police Department	Noise - Street/Sidewalk
1	New York City Police Department	Blocked Driveway
2	New York City Police Department	Blocked Driveway
3	New York City Police Department	Illegal Parking
4	New York City Police Department	Illegal Parking

	Descriptor	Location Type	Incident Zip \
0	Loud Music/Party	Street/Sidewalk	10034.0
1	No Access	Street/Sidewalk	11105.0
2	No Access	Street/Sidewalk	10458.0
3	Commercial Overnight Parking	Street/Sidewalk	10461.0
4	Blocked Sidewalk	Street/Sidewalk	11373.0

	Incident Address	... Bridge Highway Name	Bridge Highway Direction \
0	71 VERMILYEA AVENUE	...	NaN
1	27-07 23 AVENUE	...	NaN
2	2897 VALENTINE AVENUE	...	NaN
3	2940 BAISLEY AVENUE	...	NaN
4	87-14 57 ROAD	...	NaN

	Road Ramp	Bridge Highway Segment	Garage Lot	Name	Ferry Direction \
0	NaN	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN

	Ferry Terminal Name	Latitude	Longitude \
0	NaN	40.865682	-73.923501
1	NaN	40.775945	-73.915094
2	NaN	40.870325	-73.888525
3	NaN	40.835994	-73.828379
4	NaN	40.733060	-73.874170

	Location
0	(40.86568153633767, -73.92350095571744)
1	(40.775945312321085, -73.91509393898605)
2	(40.870324522111424, -73.88852464418646)
3	(40.83599404683083, -73.82837939584206)
4	(40.733059618956815, -73.87416975810375)

[5 rows x 53 columns]

--- DataFrame Columns ---

Index(['Unique Key', 'Created Date', 'Closed Date', 'Agency', 'Agency Name',
 'Complaint Type', 'Descriptor', 'Location Type', 'Incident Zip',
 'Incident Address', 'Street Name', 'Cross Street 1', 'Cross Street 2',

```
'Intersection Street 1', 'Intersection Street 2', 'Address Type',  
'City', 'Landmark', 'Facility Type', 'Status', 'Due Date',  
'Resolution Description', 'Resolution Action Updated Date',  
'Community Board', 'Borough', 'X Coordinate (State Plane)',  
'Y Coordinate (State Plane)', 'Park Facility Name', 'Park Borough',  
'School Name', 'School Number', 'School Region', 'School Code',  
'School Phone Number', 'School Address', 'School City', 'School State',  
'School Zip', 'School Not Found', 'School or Citywide Complaint',  
'Vehicle Type', 'Taxi Company Borough', 'Taxi Pick Up Location',  
'Bridge Highway Name', 'Bridge Highway Direction', 'Road Ramp',  
'Bridge Highway Segment', 'Garage Lot Name', 'Ferry Direction',  
'Ferry Terminal Name', 'Latitude', 'Longitude', 'Location'],  
dtype='object')
```

```
In [4]: # 1.4 Identify the shape of the dataset  
print(f"\nDataset Shape: {df.shape}")
```

Dataset Shape: (364558, 53)

```
In [5]: # 1.5 Identify variables with null values  
print("\n--- Missing Values Count Per Column ---")  
null_counts = df.isnull().sum()  
print(null_counts[null_counts > 0])
```

```

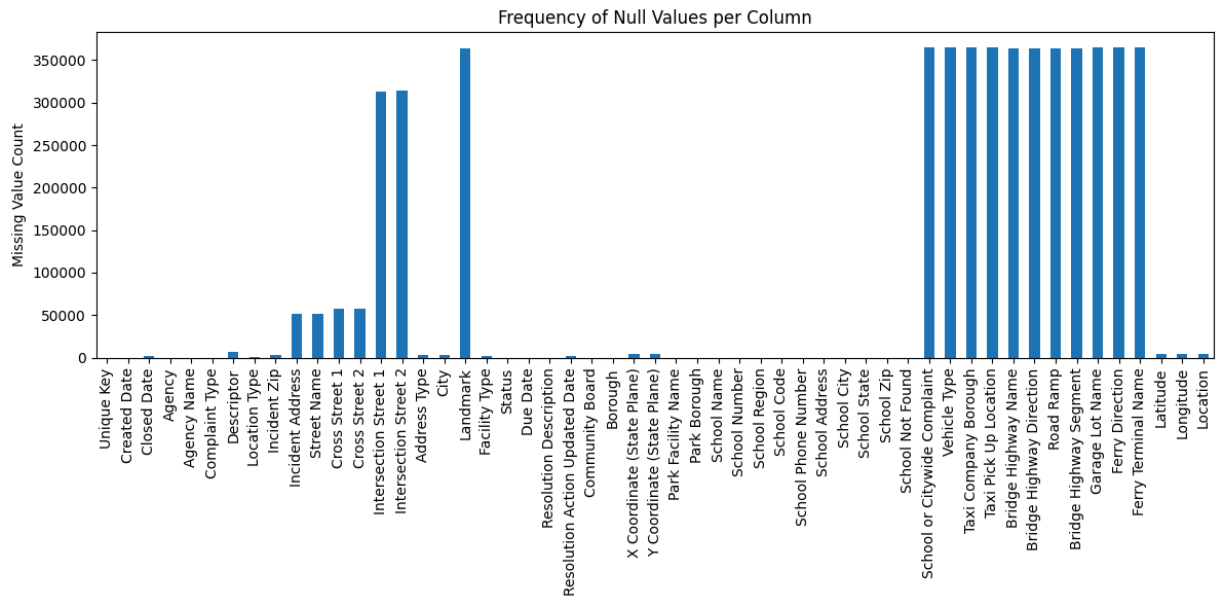
--- Missing Values Count Per Column ---
Closed Date                2381
Descriptor                 6501
Location Type              133
Incident Zip              2998
Incident Address          51699
Street Name               51699
Cross Street 1            57188
Cross Street 2            57805
Intersection Street 1     313438
Intersection Street 2     314046
Address Type              3252
City                     2997
Landmark                  364183
Facility Type             2389
Due Date                   3
Resolution Action Updated Date 2402
X Coordinate (State Plane) 4030
Y Coordinate (State Plane) 4030
School Region             1
School Code               1
School Zip                1
School or Citywide Complaint 364558
Vehicle Type              364558
Taxi Company Borough      364558
Taxi Pick Up Location      364558
Bridge Highway Name        364261
Bridge Highway Direction   364261
Road Ramp                  364296
Bridge Highway Segment     364296
Garage Lot Name            364558
Ferry Direction           364557
Ferry Terminal Name        364556
Latitude                   4030
Longitude                  4030
Location                   4030
dtype: int64

```

```

In [6]: # 2.1 Draw a frequency plot to show the number of null values
plt.figure(figsize=(12, 6))
df.isnull().sum().plot(kind='bar')
plt.title('Frequency of Null Values per Column')
plt.ylabel('Missing Value Count')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()

```



```
In [7]: # 2.2 Missing value treatment: Remove records where Closed Date is null
df = df.dropna(subset=['Closed Date'])
```

```
In [8]: # 2.3 Analyze date columns and fix incorrect timelines
df['Created Date'] = pd.to_datetime(df['Created Date'], format='%m/%d/%Y %I:%M:%S %p')
df['Closed Date'] = pd.to_datetime(df['Closed Date'], format='%m/%d/%Y %I:%M:%S %p')
```

```
In [9]: # 2.3.1 & 2.3.2 Calculate time elapsed and convert to seconds
df['Request_Closing_Time'] = (df['Closed Date'] - df['Created Date']).dt.total_seconds()

# Remove entries that have an incorrect timeline (negative time elapsed)
df = df[df['Request_Closing_Time'] >= 0]
```

```
In [10]: # 2.3.3 View descriptive statistics for the new column
print("\n--- Descriptive Statistics for Request_Closing_Time (Seconds) ---")
print(df['Request_Closing_Time'].describe())
```

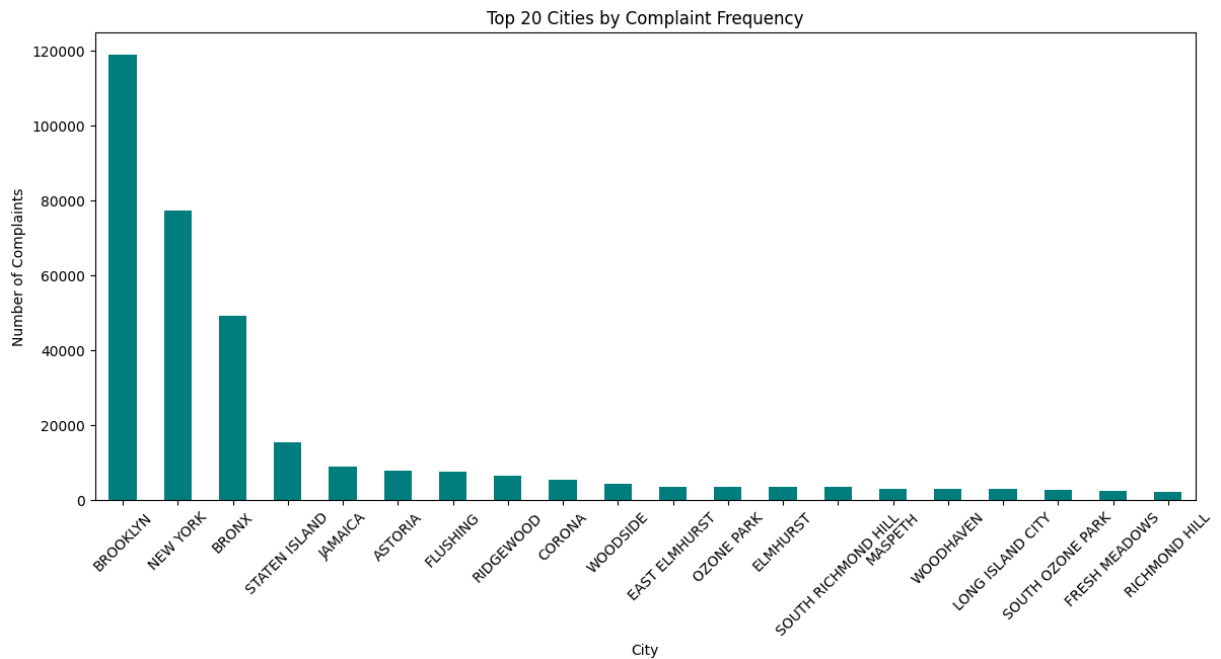
```
--- Descriptive Statistics for Request_Closing_Time (Seconds) ---
count      3.621770e+05
mean       1.511330e+04
std        2.110255e+04
min        6.100000e+01
25%        4.533000e+03
50%        9.616000e+03
75%        1.887800e+04
max        2.134342e+06
Name: Request_Closing_Time, dtype: float64
```

```
In [11]: # 2.3.4 Check null values in Complaint_Type and City
print("\nMissing values in Complaint Type:", df['Complaint Type'].isnull().sum())
print("Missing values in City:", df['City'].isnull().sum())
```

```
Missing values in Complaint Type: 0
Missing values in City: 674
```

```
In [12]: # 2.3.5 Impute the NA value in City with 'Unknown City'
df.loc[:, 'City'] = df['City'].fillna('Unknown City')
```

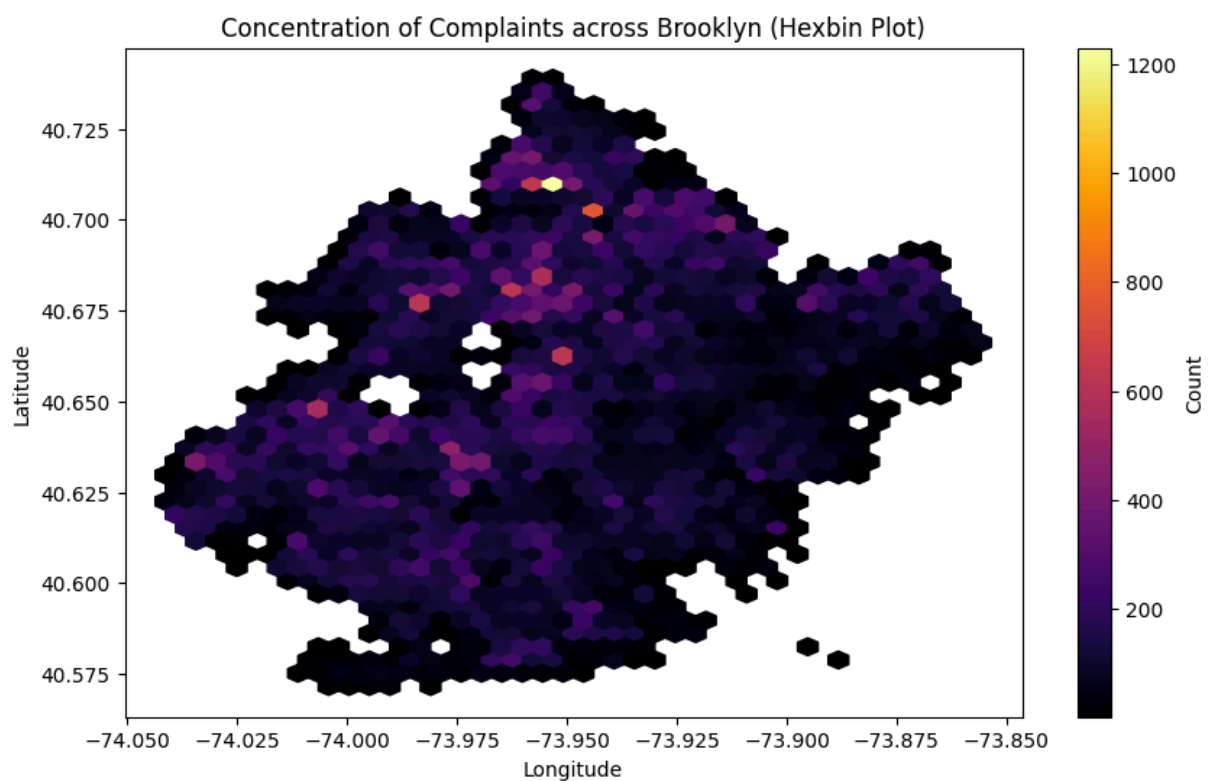
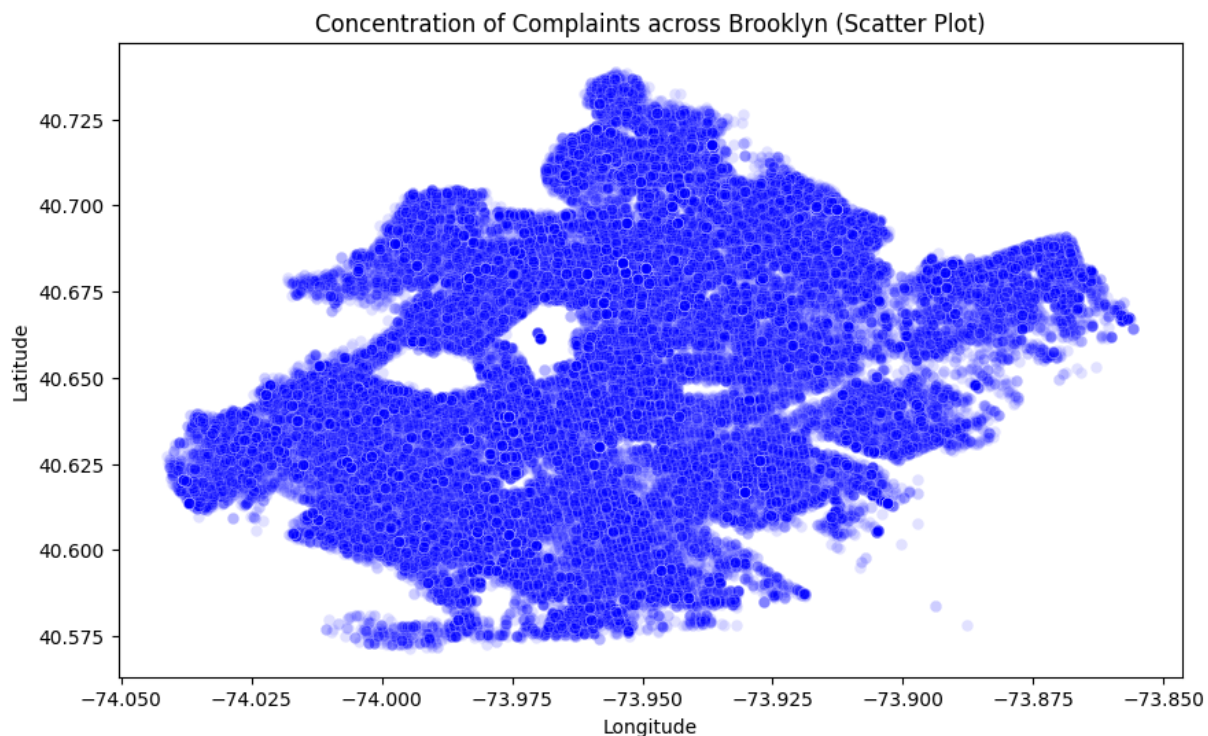
```
In [13]: # 2.3.6 Draw a frequency plot for the complaints in each city (Top 20 for readability)
plt.figure(figsize=(14, 6))
df['City'].value_counts().head(20).plot(kind='bar', color='teal')
plt.title('Top 20 Cities by Complaint Frequency')
plt.ylabel('Number of Complaints')
plt.xlabel('City')
plt.xticks(rotation=45)
plt.show()
```



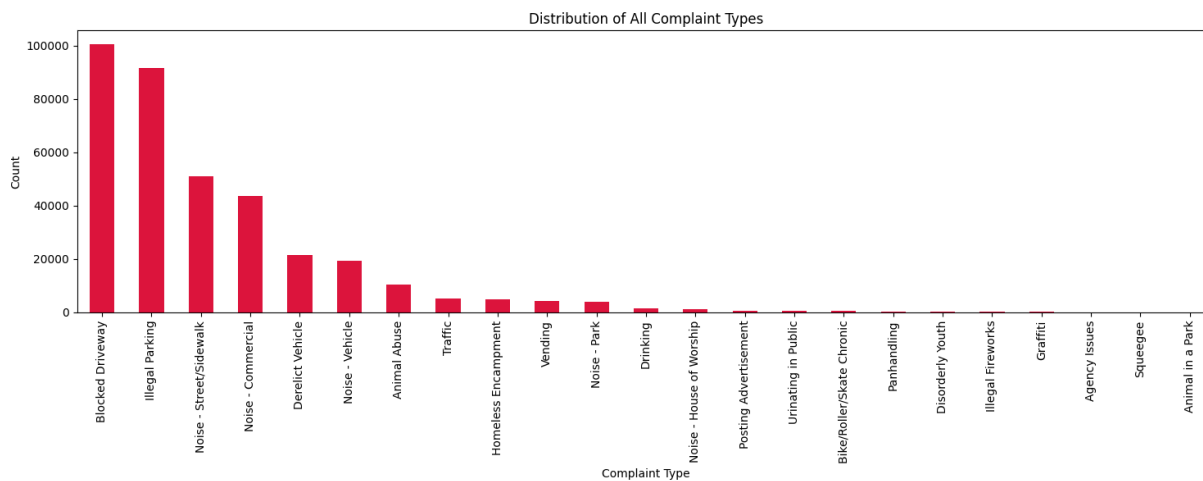
```
In [14]: # 2.3.7 Create a scatter and hexbin plot for Brooklyn concentration
df_brooklyn = df[df['Borough'] == 'BROOKLYN'].dropna(subset=['Latitude', 'Longitude'])

# Scatter plot
plt.figure(figsize=(10, 6))
sns.scatterplot(x='Longitude', y='Latitude', data=df_brooklyn, alpha=0.1, color='blue')
plt.title('Concentration of Complaints across Brooklyn (Scatter Plot)')
plt.show()

# Hexbin plot
plt.figure(figsize=(10, 6))
plt.hexbin(df_brooklyn['Longitude'], df_brooklyn['Latitude'], gridsize=40, cmap='inferno')
plt.colorbar(label='Count')
plt.title('Concentration of Complaints across Brooklyn (Hexbin Plot)')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.show()
```



```
In [15]: # 3.1 Plot a bar graph to show the types of complaints
plt.figure(figsize=(15, 6))
df['Complaint Type'].value_counts().plot(kind='bar', color='crimson')
plt.title('Distribution of All Complaint Types')
plt.xlabel('Complaint Type')
plt.ylabel('Count')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```



```
In [16]: # 3.2 Check the frequency of various types of complaints for New York City
complaint_counts = df['Complaint Type'].value_counts()
print("--- Complaint Type Frequencies ---")
print(complaint_counts)

# 3.3 Find the top 10 complaint types
top_10_complaints = df['Complaint Type'].value_counts().head(10)
print("\n--- Top 10 Complaint Types ---")
print(top_10_complaints)
```


--- Complaint Type Frequencies ---

Complaint Type	
Blocked Driveway	100624
Illegal Parking	91716
Noise - Street/Sidewalk	51139
Noise - Commercial	43751
Derelict Vehicle	21518
Noise - Vehicle	19301
Animal Abuse	10530
Traffic	5196
Homeless Encampment	4879
Vending	4185
Noise - Park	4089
Drinking	1404
Noise - House of Worship	1068
Posting Advertisement	679
Urinating in Public	641
Bike/Roller/Skate Chronic	475
Panhandling	325
Disorderly Youth	315
Illegal Fireworks	172
Graffiti	157
Agency Issues	8
Squeegee	4
Animal in a Park	1

Name: count, dtype: int64

--- Top 10 Complaint Types ---

Complaint Type	
Blocked Driveway	100624
Illegal Parking	91716
Noise - Street/Sidewalk	51139
Noise - Commercial	43751
Derelict Vehicle	21518
Noise - Vehicle	19301
Animal Abuse	10530
Traffic	5196
Homeless Encampment	4879
Vending	4185

Name: count, dtype: int64

```
In [17]: # 3.4 Display the various types of complaints in each city
city_complaints = df.groupby(['City', 'Complaint Type']).size().reset_index(name='C')
print("\n--- Breakdown of Complaints per City (Sample View) ---")
print(city_complaints.head(15))
```

--- Breakdown of Complaints per City (Sample View) ---

	City	Complaint Type	Count
0	ARVERNE	Animal Abuse	46
1	ARVERNE	Blocked Driveway	50
2	ARVERNE	Derelict Vehicle	32
3	ARVERNE	Disorderly Youth	2
4	ARVERNE	Drinking	1
5	ARVERNE	Graffiti	1
6	ARVERNE	Homeless Encampment	4
7	ARVERNE	Illegal Parking	62
8	ARVERNE	Noise - Commercial	2
9	ARVERNE	Noise - House of Worship	14
10	ARVERNE	Noise - Park	2
11	ARVERNE	Noise - Street/Sidewalk	29
12	ARVERNE	Noise - Vehicle	10
13	ARVERNE	Panhandling	1
14	ARVERNE	Traffic	1

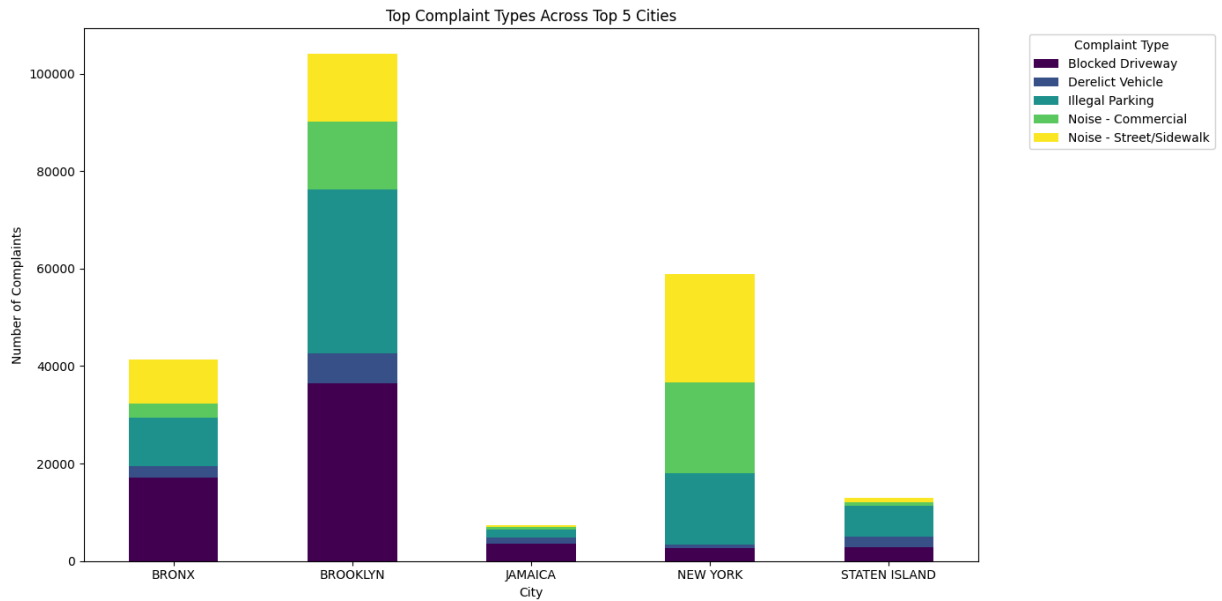
```
In [18]: # 3.5 Create a DataFrame, df_new, which contains cities as columns and complaint ty
df_new = df.groupby(['Complaint Type', 'City']).size().unstack(fill_value=0)
print("\n--- Transformed DataFrame (df_new) Shape ---")
print(df_new.shape)
```

--- Transformed DataFrame (df_new) Shape ---
(23, 54)

```
In [19]: # 4.1 Stacked/Grouped bar chart showing the major types of complaints in each city
top_cities = df['City'].value_counts().head(5).index
top_types = df['Complaint Type'].value_counts().head(5).index

df_filtered = df[df['City'].isin(top_cities) & df['Complaint Type'].isin(top_types)]
df_plot = df_filtered.groupby(['City', 'Complaint Type']).size().unstack(fill_value=0)

df_plot.plot(kind='bar', stacked=True, figsize=(14, 7), cmap='viridis')
plt.title('Top Complaint Types Across Top 5 Cities')
plt.xlabel('City')
plt.ylabel('Number of Complaints')
plt.xticks(rotation=0)
plt.legend(title='Complaint Type', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

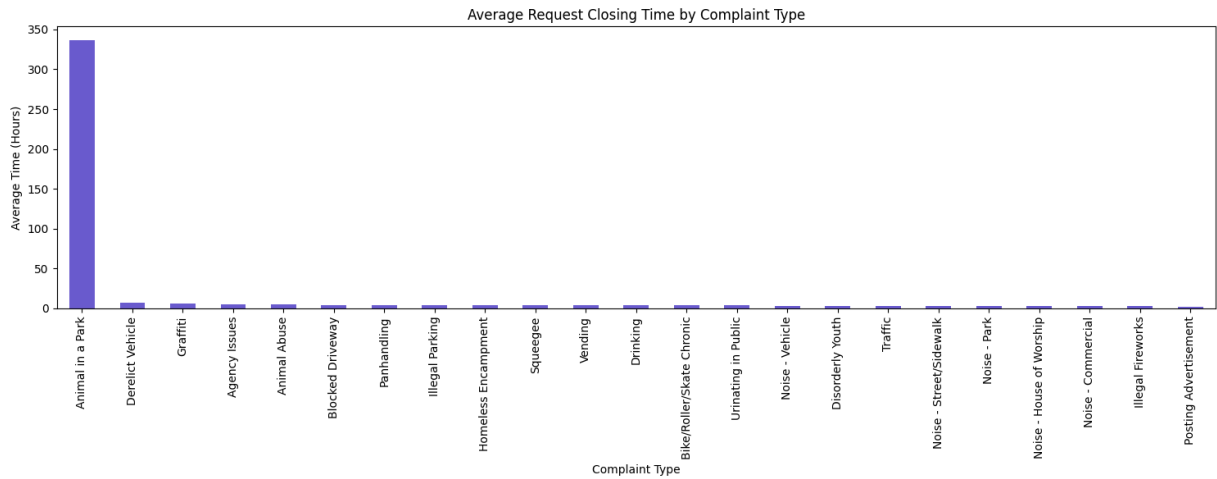


```
In [20]: # 4.2 Sort the complaint types based on the average Request_Closing_Time grouping t
avg_closing_time = df.groupby(['City', 'Complaint Type'])['Request_Closing_Time'].m
avg_closing_time_sorted = avg_closing_time.sort_values(by=['City', 'Request_Closing

print("\n--- Sorted Average Request Closing Time per Location (Sample) ---")
print(avg_closing_time_sorted.head(15))
```

```
--- Sorted Average Request Closing Time per Location (Sample) ---
   City      Complaint Type  Request_Closing_Time
4  ARVERNE      Drinking              859.000000
16 ARVERNE      Vending              1735.000000
15 ARVERNE  Urinating in Public              2491.000000
13 ARVERNE      Panhandling              3673.000000
14 ARVERNE      Traffic              4014.000000
10 ARVERNE  Noise - Park              4638.000000
5  ARVERNE      Graffiti              5508.000000
12 ARVERNE  Noise - Vehicle              5673.600000
6  ARVERNE  Homeless Encampment              6541.250000
9  ARVERNE  Noise - House of Worship              6653.428571
11 ARVERNE  Noise - Street/Sidewalk              7173.206897
8  ARVERNE  Noise - Commercial              8234.000000
1  ARVERNE  Blocked Driveway              8318.840000
0  ARVERNE      Animal Abuse              8399.195652
7  ARVERNE  Illegal Parking              8406.080645
```

```
In [21]: # 5.1 Visualize the average of Request_Closing_Time across different complaint type
plt.figure(figsize=(15, 6))
# Calculating average closing time in hours for better readability
(df.groupby('Complaint Type')['Request_Closing_Time'].mean() / 3600).sort_values(as
plt.title('Average Request Closing Time by Complaint Type')
plt.xlabel('Complaint Type')
plt.ylabel('Average Time (Hours)')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```



```
In [22]: # 6 & 7: Statistical Analysis using p-values (Kruskal-Wallis H test)
print("--- Launching Statistical Hypothesis Testing ---")

# Structure data groups for the test: drop missing times and group by complaint type
df_stats = df.dropna(subset=['Request_Closing_Time', 'Complaint Type'])
groups = [group['Request_Closing_Time'].values for name, group in df_stats.groupby('Complaint Type')]

# Run the Kruskal-Wallis H Test
stat, p_value = stats.kruskal(*groups)

print(f"Kruskal-Wallis H-statistic: {stat:.4f}")
print(f"P-value: {p_value}")

--- Launching Statistical Hypothesis Testing ---
Kruskal-Wallis H-statistic: 11988.2694
P-value: 0.0
```

```
In [23]: # 7.1 & 7.2 Evaluate the Hypothesis
alpha = 0.05
if p_value < alpha:
    print("\nResult: Reject the Null Hypothesis (H0).")
    print("Conclusion: The average response times across different complaint types are significantly different.")
else:
    print("\nResult: Fail to reject the Null Hypothesis (H0).")
    print("Conclusion: There is no statistical difference in average response times across different complaint types.")
```

Result: Reject the Null Hypothesis (H0).

Conclusion: The average response times across different complaint types are significantly DIFFERENT.

Step 8: Observations

- Dominant complaint types and cities identified.
- Average closing time differences visualized.
- Statistical significance tested with Kruskal-Wallis.

In []: